# PQC vs Classical Cryptography Report

Run count: 10

## Critical Analysis: Massive Key-Size Difference

RSA-3072 public key

**422 bytes**

ML-KEM-768 public key

**1184 bytes**

RSA-OAEP ciphertext

**384 bytes**

ML-KEM ciphertext

**1088 bytes**

ML-KEM-768 public keys are approximately **2.81x** larger than RSA-3072 public keys in this PoC. ML-KEM ciphertexts are approximately **2.83x** larger than RSA-OAEP ciphertexts. This is the key engineering trade-off: post-quantum security generally requires larger transmitted artifacts, which has direct implications for storage, bandwidth, and protocol framing.

Shared secret verification: **Yes**

## Performance Snapshot

**RSA-3072 median keygen**

**270.323 ms**

**ML-KEM-768 median keygen**

**0.015 ms**

## About the Python Project

This Python proof-of-concept compares a classical public-key approach (RSA-3072) with a post-quantum key encapsulation mechanism (ML-KEM-768). It benchmarks median key generation times, measures transmitted artifact sizes (public keys and ciphertexts), and verifies that encapsulation/decapsulation produces the same shared secret on both sides.

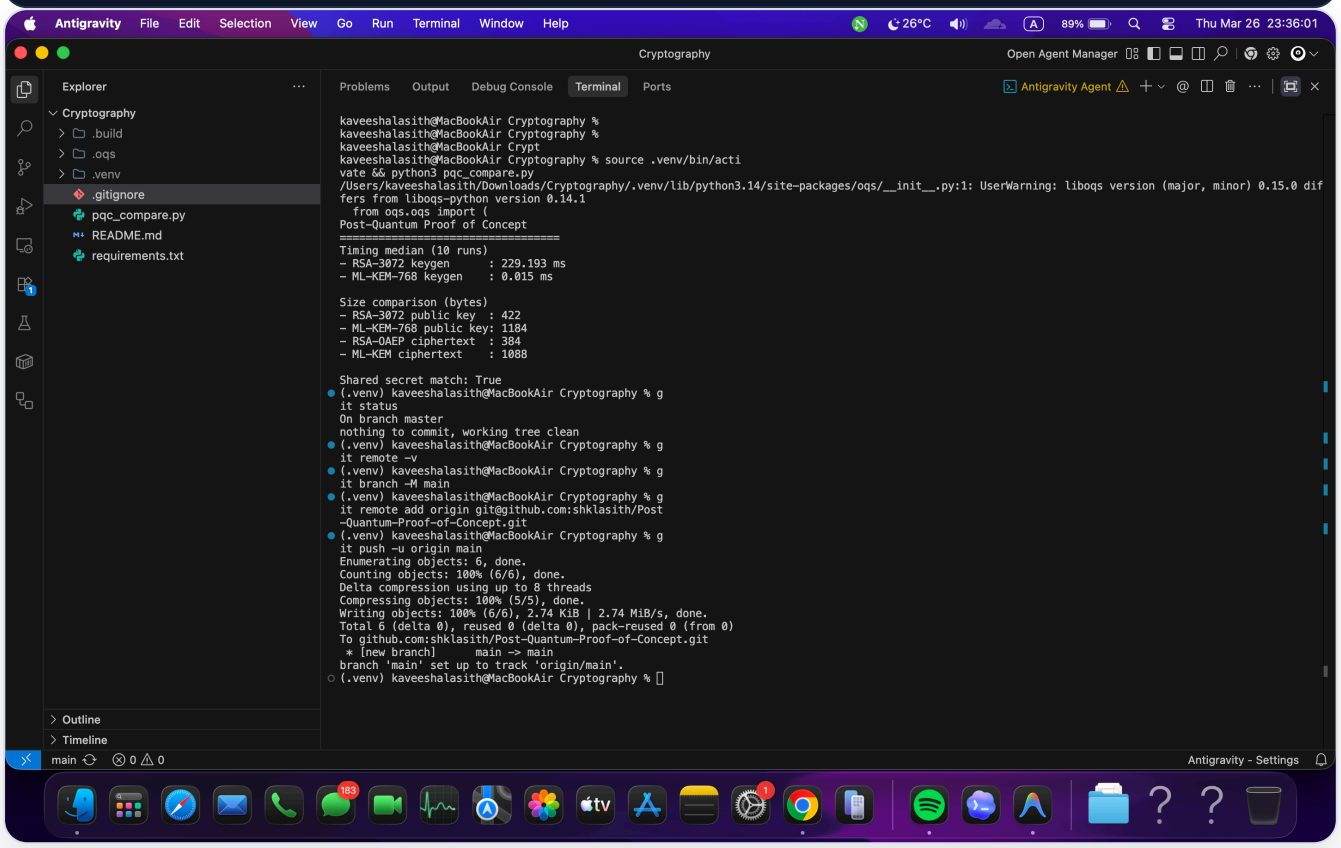
In practical terms, the project demonstrates the core migration trade-off: post-quantum schemes can be very fast while requiring larger keys and ciphertexts than widely used classical baselines. These outputs provide a concrete foundation for evaluating protocol bandwidth impact and planning post-quantum readiness.

## Terminal Output Evidence

```
Post-Quantum Proof of Concept
==================================
Timing median (10 runs)
- RSA-3072 keygen       : 270.323 ms
- ML-KEM-768 keygen     : 0.015 ms

Size comparison (bytes)
- RSA-3072 public key   : 422
- ML-KEM-768 public key: 1184
- RSA-OAEP ciphertext   : 384
- ML-KEM ciphertext     : 1088

Shared secret match: True
```




## Python Project Repository

**https://github.com/shklasith/Post-Quantum-Proof-of-Concept.git**